ActiveRecord::Base is the most important class in Active Record. Not only does it contain class methods that do important things, such as opening database connections, it's also the superclass of all mapped classes, such as Duck:

```
class Duck < ActiveRecord::Base
  validate do
    errors.add(:base, "Illegal duck name.") unless name[0] == 'D'
  end
end
```

The validate method is a *Class Macro (117)* that takes a block. You don't have to worry about the details of the code in the block—just know that in this example, it ensures that a Duck's name begins with a D. (Our company's duck-naming policies demand that.) If you try to save a Duck with an illegal name to the database, the save! method will raise an exception, while the more discreet save will fail silently.

By convention, Active Record automatically maps Duck objects to the ducks table. By looking at the database schema, Active Record also finds out that Ducks have a name, and it defines a *Ghost Method (57)* to access that field. Thanks to these conventions, you can use the Duck class right away:

```
my_duck = Duck.new
my_duck.name = "Donald"
my_duck.valid?         # => true
my_duck.save!
```

I've checked that my_duck is valid (it begins with a D) and saved it to the database. Reading it back, you get this:

```
duck_from_database = Duck.first
duck_from_database.name         # => "Donald"
duck_from_database.delete
```

That's enough code for now to give you a sense of how Active Record is meant to be used. Now let's see what's happening under the hood.

How Active Record Is Put Together

The code in the previous example looks simple, but ActiveRecord::Base is capable of much more than that. Indeed, the more you use Active Record, the more the methods in Base seem to multiply. You might assume that Base is a huge class with thousands of lines of code that define methods such as save or validate.

Surprisingly, the source code of ActiveRecord::Base contains no trace of those methods. This is a common problem for newcomers to Rails: it's often difficult